

```

Party notary = getServiceHub().getNetworkMapCache().
getNotaryIdentities().get(0);

// We create the transaction components.
IOUState outputState = new IOUState(iouValue,
getOurIdentity(), otherParty);
Command command = new Command<>(new TemplateContract.
Commands.Action(), getOurIdentity().getOwningKey());

// We create a transaction builder and add the
components.
TransactionBuilder txBuilder = new
TransactionBuilder(notary)
    .addOutputState(outputState,
TemplateContract.ID)
    .addCommand(command);

// Signing the transaction.
SignedTransaction signedTx = getServiceHub().signInit
ialTransaction(txBuilder);

// Creating a session with the other party.
FlowSession otherPartySession =
initiateFlow(otherParty);

// We finalise the transaction and then send it to the
counterparty.

```

```

subFlow(new FinalityFlow(signedTx,
otherPartySession));

return null;
}
}

```

The corresponding Kotlin code is also available. The complete *Hello World* tutorial along with instructions on how to modify and execute it is provided at <https://docs.corda.net/hello-world-introduction.html>.

This article has provided only an introduction to Corda. The quantum of resources available with Corda is larger. If you are interested in the blockchain, try exploring Corda for features that match your requirements. **END** 🐧

References

- [1] <https://www.corda.net/>
- [2] <https://docs.corda.net/index.html#>
- [3] <https://github.com/corda/corda>

By: Dr K.S. Kuppusamy

The author is an assistant professor of computer science, School of Engineering and Technology, Pondicherry Central University. He has 13+ years of teaching and research experience in academia and industry. He can be reached via mail at kskuppu@gmail.com.

Continued from Page 86...

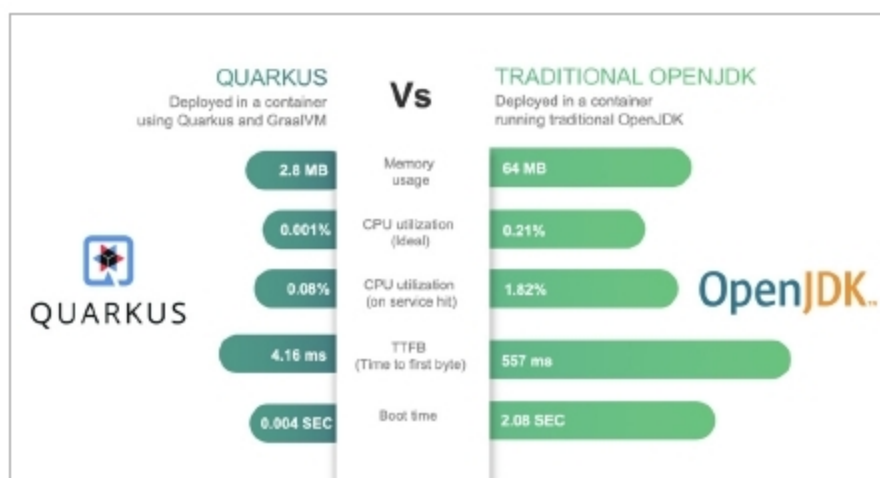


Figure 7: Comparing Quarkus-GraalVM with a traditional deployment

On hitting the service <http://localhost:8080/hello/person>, the increase in CPU usage was 0.08 per cent, which was marginally less compared to the previous test. So was the TTFB collected by the Chrome dev tool (Figure 6).

The TTFB was 4.16ms compared to 557ms, which was 98 per cent less than the last test.

The experiments I have shared are just a subset of the other scenarios I tested. But the last experiment is one of the most important, according to me, if we are going to try migrating our systems to Quarkus. It gave me a lot

of confidence to migrate old applications running on traditional systems to Quarkus, and I am definitely going to give it a try. I also ran a load test to see how things hold under load. So far, I have only had great results with the metrics. Though there are a few different scenarios that I didn't try, I believe that based on what was tested, the migration to Quarkus should definitely be tried, and great results can be accomplished by doing so. **END** 🐧

References

- [1] Quarkus, website: <https://quarkus.io>
- [2] GraalVM: <https://www.graalvm.org/>
- [3] Quarkus release announcement: <https://developers.redhat.com/blog/2019/03/07/quarkus-next-generation-kubernetes-native-java-framework/>

By: Jaydeep Chakrabarty

The author works at Thoughtworks Technologies, India. He has contributed to the open source community for the last 14 years. Among the various open source projects across tech-stacks that he has contributed to are Talisman, OpenStack projects, Report-mine, Taiko, etc. He can be reached at jaydeepc@thoughtworks.com.